



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 7
по курсу «Языки и методы программирования»
«GUI сабскрайбер и ридер MQTT»

Студент группы ИУ9-21Б Белянин Г. И.

Преподаватель Посевин Д. П.

Москва 2026

1 Цель

Изучить принципы работы протокола MQTT в клиент-серверной архитектуре и приобрести навыки разработки распределенных графических приложений на Java. Освоить механизмы асинхронного обмена сообщениями между удаленными узлами (издатель/подписчик) с визуализацией процессов публикации и получения данных через пользовательский интерфейс.

2 Персональный вариант

Разбивка на команды по 2 человека

На одном ПК GUI сабскрайбер, на втором брокер и GUI ридер. Сабскрайбер читает, ридер пишет — все через GUI реализовать. (Выполнял в паре с Тумгоевым А-М.)

3 Практическая реализация

Класс SenderForm

```
package org.example;

import org.eclipse.paho.client.mqttv3.*;
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class SenderForm extends JFrame {
    private JPanel mainPanel;
    private JTextField messageField;
    private JButton sendButton;

    private MqttClient client;

    public SenderForm() {
        setTitle("GuiSender");
        setContentPane(mainPanel);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(400, 150);
        setLocationRelativeTo(null);

        try {
            client = new MqttClient("tcp://172.20.10.2:1883", "GuiSender");
            MqttConnectOptions options = new MqttConnectOptions();
            options.setCleanSession(true);
            client.connect(options);
            System.out.println("Connected successfully!");
        } catch (MqttException e) {
            JOptionPane.showMessageDialog(this, "Connection failed: " + e.getMessage());
        }

        sendButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                try {
                    String textToSend = messageField.getText();

                    if (!textToSend.trim().isEmpty()) {
                        MqttMessage message = new MqttMessage(textToSend.getBytes());
                        client.publish("chat/messages", message);
                    }
                } catch (Exception ex) {
                    System.out.println("Error: " + ex.getMessage());
                }
            }
        });
    }
}
```

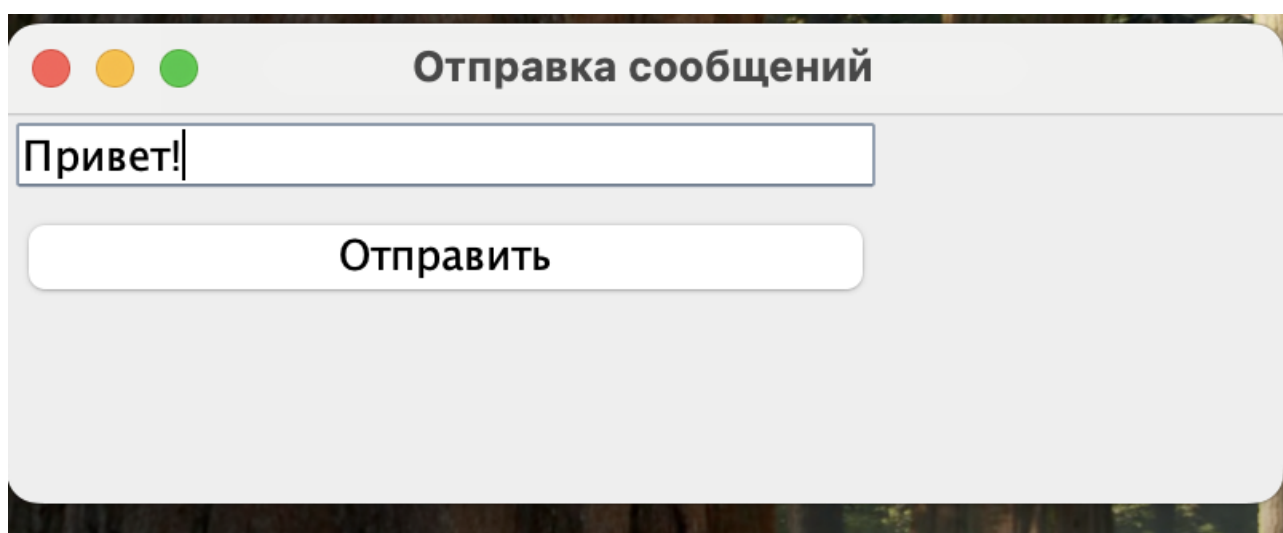
```

        messageField.setText(""); //
    }
} catch (MqttException ex) {
    JOptionPane.showMessageDialog(SenderForm.this, "
        !");
}
}
});
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new SenderForm().setVisible(true));
}
}

```

4 Результаты



В результате работы программы получился следующий вывод:

5 Выводы

В ходе лабораторной работы было разработано распределенное приложение, реализующее паттерн «Издатель-Подписчик» (Publisher-Subscriber) на базе протокола MQTT. Архитектура: Реализована двухузловая система, где один экземпляр приложения выступает в роли GUI-клиента для публикации сообщений (Reader/Publisher), а другой — для их приема и отображения (Subscriber), взаимодействуя через общий MQTT-брокер. Техническая реализация: Инте-

гирована MQTT-библиотека (например, Eclipse Paho) с графическим интерфейсом (Swing/JavaFX). Освоены навыки обработки сетевых событий в отдельном потоке исполнения, чтобы избежать блокировки основного потока GUI при ожидании сообщений. Результат: Успешно проверена сквозная передача данных между разными хостами. Подтверждена корректность работы топиков MQTT и способность интерфейса динамически обновляться при поступлении новых данных от удаленного отправителя. Работа позволила закрепить понимание принципов слабого связывания компонентов в распределенных системах и особенностей реализации сетевого взаимодействия в десктопных приложениях.